



GenAI Developer

Complete Training Curriculum

Python • Machine Learning • NLP • LangChain • LangGraph • MCP • Prompt Engineering

Industry-Standard | From Zero to Production | 2025 Edition

 Modules 8 Phases	 Duration 6–8 Months	 Tools 15+ Tools	 Projects 12+ Projects	 Goal Job Ready
--	---	---	---	--

About This Curriculum

This curriculum is designed to take a complete fresher — someone with no prior experience in AI, Machine Learning, or programming — and transform them into a job-ready Generative AI Developer aligned with current industry demands (2025). Each phase builds upon the previous one in a structured, hands-on manner, blending theory with real-world projects.

Audience	Complete freshers with basic computer literacy. No prior programming or ML experience required.
Approach	Theory (30%) + Hands-on Coding (50%) + Projects & Review (20%). Every phase ends with a mini-project.
Tech Stack	Python, Jupyter, VS Code, LangChain, LangGraph, MCP, OpenAI API, HuggingFace, FastAPI, Docker, Git
Outcome	Ability to build production-grade GenAI applications, RAG pipelines, AI Agents, and MCP-integrated systems.

Curriculum Roadmap at a Glance

Phase	Module	Focus Area	Duration
1	Python Foundations	Variables, OOP, File I/O, Libraries (NumPy, Pandas, Matplotlib)	3–4 Weeks
2	Math & Statistics	Linear Algebra, Probability, Statistics, Calculus Basics	2–3 Weeks
3	Machine Learning	Supervised, Unsupervised, Model Evaluation, Scikit-learn	4–5 Weeks
4	NLP & Deep Learning	Text Processing, Transformers, BERT, Hugging Face	3–4 Weeks
5	Generative AI Core	LLMs, GPT, API Usage, RAG, Vector DBs, Fine-tuning	4–5 Weeks
6	Prompt Engineering	Zero-shot, Few-shot, CoT, ReAct, System Prompts, Evals	2–3 Weeks
7	LangChain & LangGraph	Chains, Agents, Memory, Tools, Stateful Graphs, Multi-Agent	4–5 Weeks
8	MCP & Production	Model Context Protocol, FastAPI, Docker, MLOps, Deployment	3–4 Weeks

PHASE 1 Python Programming Foundations 3–4 Weeks

Python is the backbone of all AI/ML development. This phase ensures the trainee becomes proficient in Python before touching any AI concept.

#	Topic	Key Concepts & Sub-topics
1	Environment Setup	• Install Python 3.11+, pip, conda • VS Code / PyCharm setup & extensions • Jupyter Notebook & Jupyter Lab • Virtual environments (venv, conda env) • Git basics: init, commit, push, pull, branching
2	Python Basics	• Variables, data types, type casting • Operators: arithmetic, comparison, logical, bitwise • Strings: slicing, formatting (f-strings), methods • Input/output, print formatting • Comments, docstrings, code style (PEP-8)
3	Control Flow	• if / elif / else conditions • for loops, while loops, break, continue, pass • List comprehensions, dict comprehensions • Nested loops and conditionals • Ternary operators
4	Data Structures	• Lists: creation, indexing, slicing, methods • Tuples: immutability, packing/unpacking • Dictionaries: CRUD, nested dicts, dict methods • Sets: union, intersection, difference • Stacks, queues using collections (deque)
5	Functions & Scope	• Defining functions, parameters, return values • Default arguments, *args, **kwargs • Lambda functions, map, filter, reduce • Variable scope: local, global, nonlocal • Decorators (basics) and closures
6	OOP in Python	• Classes, objects, __init__, self • Inheritance, super(), method overriding • Encapsulation: public, protected, private • Polymorphism, dunder/magic methods • Abstract classes, dataclasses, @property
7	Error Handling	• try, except, else, finally blocks • Custom exceptions, raising exceptions • Logging module (info, warning, error) • Debugging with pdb, VS Code debugger • Unit testing with pytest
8	File & I/O	• Read/write text files, CSV, JSON • os and pathlib modules • Working with directories, glob patterns • pickle for object serialization

		• Reading from APIs using requests library
9	Python Libraries	• NumPy: arrays, operations, broadcasting, indexing • Pandas: DataFrame, Series, merge, groupby, pivot • Matplotlib & Seaborn: line, bar, scatter, heatmap plots • Plotly: interactive visualizations • datetime, re (regex), itertools, functools

 Mini Project	Build a CLI-based Data Analysis Tool — load a CSV, perform EDA (summary stats, null analysis, visualizations), and export a report using pandas + matplotlib.
--	---

PHASE 2 **Mathematics & Statistics for AI/ML** ⌚ 2–3 Weeks

A solid mathematical foundation is critical for understanding how ML algorithms work under the hood. Focus on intuition, not rote formulas.

#	Topic	Key Concepts & Sub-topics
1	Linear Algebra	- Scalars, vectors, matrices, tensors - Matrix multiplication, transpose, inverse - Dot product, cross product - Eigenvalues & eigenvectors (PCA intuition) - Norms (L1, L2), distance metrics (Euclidean, Cosine)
2	Calculus Basics	- Derivatives and partial derivatives - Chain rule (core of backpropagation) - Gradient and gradient descent intuition - Minima/maxima — loss function landscapes - Learning rate effect on convergence
3	Probability	- Basic probability, conditional probability, Bayes' theorem - Probability distributions: Normal, Bernoulli, Binomial, Poisson - Joint, marginal, and conditional distributions - Expectation, variance, standard deviation - Law of large numbers, Central Limit Theorem
4	Statistics	- Descriptive stats: mean, median, mode, IQR, skewness - Hypothesis testing: p-value, t-test, chi-square - Correlation vs. causation, Pearson/Spearman - Confidence intervals, significance levels - A/B testing fundamentals
5	Optimization	- Gradient Descent: batch, mini-batch, stochastic - Momentum, RMSProp, Adam optimizer intuition - Convex vs. non-convex functions - Regularization: L1 (Lasso), L2 (Ridge) math intuition - Cross-entropy loss, MSE loss functions



Mini Project

Implement Gradient Descent from scratch in Python (no libraries) and visualize the loss curve for a linear regression problem.

PHASE 3 Machine Learning Fundamentals 4–5 Weeks

Core ML concepts that every GenAI developer must understand — model behavior, evaluation, overfitting, and real-world application of classical algorithms.

#	Topic	Key Concepts & Sub-topics
1	ML Pipeline	- Data collection, cleaning, preprocessing - Feature engineering and selection - Train/validation/test split strategies - Cross-validation (k-fold, stratified) - Scikit-learn Pipeline and ColumnTransformer
2	Supervised Learning	- Linear Regression — assumptions, implementation - Logistic Regression — binary & multiclass - Decision Trees — Gini, entropy, max depth - Random Forest — bagging, feature importance - Gradient Boosting: XGBoost, LightGBM, CatBoost
3	Unsupervised Learning	- K-Means Clustering — elbow method - Hierarchical Clustering — dendrograms - DBSCAN — density-based clustering - PCA — dimensionality reduction, variance explained - t-SNE & UMAP — high-dim visualization
4	Model Evaluation	- Classification metrics: Accuracy, Precision, Recall, F1, AUC-ROC - Regression metrics: RMSE, MAE, R^2 - Confusion matrix analysis - Bias-variance tradeoff - Overfitting / underfitting — diagnosis and fixes
5	Optimization & Tuning	- Hyperparameter tuning: GridSearchCV, RandomizedSearchCV - Optuna for automated hyperparameter optimization - Feature scaling: StandardScaler, MinMaxScaler, RobustScaler - Handling imbalanced data: SMOTE, class_weight - Ensemble methods: Voting, Stacking, Blending
6	ML Tools & MLflow	- Scikit-learn complete workflow - MLflow: experiment tracking, model registry - Joblib / Pickle for model serialization - SHAP for model explainability - Weights & Biases (W&B) for experiment logging



Mini Project

End-to-end ML Project: Customer Churn Prediction — EDA → Feature Engineering → Model Training (XGBoost) → Evaluation → SHAP Explainability → MLflow tracking.

PHASE 4 **NLP & Deep Learning Foundations** ⌚ 3–4 Weeks

Understanding how text is processed and how neural networks learn is the gateway to understanding Large Language Models and Generative AI.

#	Topic	Key Concepts & Sub-topics
1	NLP Fundamentals	• Tokenization: word, sentence, subword • Text cleaning: stopwords, stemming, lemmatization • Bag of Words (BoW), TF-IDF • N-grams, vocabulary building • spaCy and NLTK libraries
2	Word Embeddings	• Word2Vec: CBOW and Skip-gram • GloVe embeddings • FastText for subword embeddings • Cosine similarity between embeddings • Sentence embeddings overview
3	Neural Networks	• Perceptron, multilayer perceptron (MLP) • Activation functions: ReLU, Sigmoid, Tanh, GELU • Forward pass, backward pass, weight updates • Batch normalization, dropout • PyTorch basics: tensors, autograd, nn.Module
4	Sequence Models	• Recurrent Neural Networks (RNNs) — vanishing gradient • LSTMs and GRUs — gating mechanisms • Encoder-Decoder architecture • Sequence-to-sequence models • Attention mechanism (additive, multiplicative)
5	Transformer Architecture	• Self-attention and multi-head attention • Positional encoding • Encoder stack vs. Decoder stack • Layer normalization, feed-forward layers • BERT (encoder-only), GPT (decoder-only), T5 (encoder-decoder)
6	Hugging Face Ecosystem	• transformers library: AutoModel, AutoTokenizer, pipeline() • Datasets library for NLP datasets • Text classification, NER, summarization, translation with pipelines • Tokenizer deep-dive: padding, truncation, attention masks • Model cards and hub usage



Mini Project

Build a Sentiment Analysis system using BERT (HuggingFace) with fine-tuning on a custom dataset. Evaluate with classification report and visualize attention weights.

PHASE 5 Generative AI — Core Concepts 🕒 4–5 Weeks

The heart of the curriculum. This phase covers everything needed to work with LLMs in production — from API usage to RAG systems and fine-tuning.

#	Topic	Key Concepts & Sub-topics
1	LLM Fundamentals	- What are Large Language Models (LLMs)? - Pre-training, fine-tuning, RLHF - Tokens, context window, vocabulary size - Temperature, top-k, top-p sampling - GPT-4, Claude, Gemini, Llama, Mistral — comparison
2	OpenAI / LLM APIs	- OpenAI API: chat completions, embeddings - System, user, assistant message roles - Function calling / Tool calling - Streaming responses, async calls - Token cost estimation, rate limiting, error handling
3	Embeddings & Similarity	- text-embedding-ada-002 and newer embedding models - Cosine similarity for semantic search - Embedding-based classification - Clustering documents using embeddings - Evaluating embedding quality
4	Vector Databases	- What is a Vector DB? Why do LLMs need it? - FAISS: index types (Flat, HNSW, IVF), similarity search - ChromaDB: collections, upsert, query - Pinecone: cloud-native vector store - Qdrant, Weaviate — overview and comparison
5	RAG Systems	- Retrieval-Augmented Generation (RAG) architecture - Chunking strategies: fixed, recursive, semantic - Document loaders: PDF, DOCX, HTML, CSV - Hybrid search: dense + sparse (BM25) - Advanced RAG: HyDE, Self-RAG, Corrective RAG, Contextual Retrieval
6	Fine-tuning LLMs	- When to fine-tune vs. prompt engineer vs. RAG - Supervised Fine-Tuning (SFT) on task-specific data - LoRA and QLoRA — parameter-efficient fine-tuning - PEFT library: LoraConfig, get_peft_model - Training with Hugging Face Trainer / trl library
7	Open Source LLMs	- Llama 3, Mistral, Phi-3, Gemma model families - Running models locally with Ollama - GGUF format, llama.cpp quantization - LM Studio for local model testing - Quantization: INT4, INT8, GPTQ, AWQ
8	Multimodal AI	- Vision Language Models (VLMs): GPT-4o, LLaVA, Gemini Vision - Image understanding with LLMs - Audio models: Whisper for transcription - Text-to-image: Stable Diffusion, DALL-E basics

		• Document AI: parsing complex PDFs/images with LLMs
--	--	--

 Mini Project

Build a full RAG application: ingest a PDF knowledge base → chunk and embed → store in ChromaDB → query with GPT-4 → return grounded answers with source citations.

PHASE 6 Prompt Engineering Mastery ⌚ 2–3 Weeks

Prompt engineering is both a science and an art. This phase teaches systematic techniques to elicit the best outputs from any LLM, and to evaluate prompt quality rigorously.

#	Topic	Key Concepts & Sub-topics
1	Prompt Basics	- Anatomy of a prompt: instruction, context, input, output format - Zero-shot prompting - One-shot and few-shot prompting - Role prompting (persona assignment) - Instruction tuning principles
2	Advanced Techniques	- Chain-of-Thought (CoT) prompting - Zero-shot CoT: 'Let's think step by step' - Tree of Thought (ToT) — branching reasoning - ReAct (Reasoning + Acting) pattern - Self-consistency prompting
3	System Prompts	- System prompt design principles - Persona, tone, and format control - Guardrails and constraints in prompts - Output format control: JSON, Markdown, XML - Grounding and context injection strategies
4	Structured Output	- Forcing JSON output with OpenAI JSON mode - Pydantic models for output validation - Instructor library for structured extraction - Response validation and retry logic - Schema design for LLM outputs
5	Prompt Patterns	- Template Pattern, Persona Pattern - Question Refinement Pattern - Cognitive Verifier Pattern - Output Automator Pattern - Meta Language Creation Pattern
6	Prompt Evaluation	- Manual evaluation: coherence, faithfulness, relevance - Automated eval with LLM-as-Judge - RAGAs framework for RAG evaluation - G-Eval and MT-Bench benchmarks - Building an eval dataset and scoring pipeline
7	Prompt Security	- Prompt injection attacks and defenses - Jailbreaking risks and mitigations - Input sanitization for LLM apps - PII handling in prompts - Safe deployment practices



Mini Project

Build a Prompt Optimization Toolkit — systematically test 5 prompt variants on the same task, score with LLM-as-Judge, visualize results, and recommend the best prompt.

PHASE 7 LangChain & LangGraph — AI Application Framework 4–5 Weeks

LangChain and LangGraph are the industry-standard frameworks for building LLM-powered applications and agents. This is the most hands-on phase with extensive coding.

7A — LangChain Deep Dive

#	Topic	Key Concepts & Sub-topics
1	LangChain Setup	- Installation, environment setup, LangSmith tracing - LangChain v0.3 architecture overview - Core abstractions: Runnable, RunnableSequence - LCEL (LangChain Expression Language) — pipe syntax - LangSmith for debugging and observability
2	Models & Prompts	- ChatOpenAI, ChatAnthropic, ChatOllama, HuggingFace - PromptTemplate, ChatPromptTemplate, MessagesPlaceholder - Partial prompts, prompt composition - Output parsers: StrOutputParser, PydanticOutputParser - with_structured_output() for type-safe outputs
3	Chains with LCEL	- Simple chains: prompt llm parser - RunnablePassthrough, RunnableParallel - RunnableLambda for custom functions - Branching chains with RunnableBranch - Retry logic and fallback chains
4	Document Handling	- Document loaders: PyPDFLoader, WebBaseLoader, CSVLoader - Text splitters: RecursiveCharacterTextSplitter, SemanticChunker - Embedding models in LangChain - VectorStore integration: Chroma, FAISS, Pinecone - Retriever types: similarity, MMR, threshold-based
5	RAG Chains	- Basic RAG chain: retriever prompt llm - Conversational RAG with history awareness - Multi-query retriever for better recall - Contextual compression retriever - Ensemble retrievers (dense + sparse)
6	Memory	- ConversationBufferMemory, ConversationSummaryMemory - ConversationBufferWindowMemory (sliding window) - ChatMessageHistory for persistent history - InMemoryChatMessageHistory vs external (Redis, DynamoDB) - Memory in LCEL chains
7	Tools & Toolkits	@tool decorator, StructuredTool - Built-in tools: TavilySearchResults, Wikipedia, ArXiv - Custom tool creation with Pydantic input schemas - Toolkits: SQLDatabaseToolkit, FileManagementToolkit - Tool error handling and validation

8	LangChain Agents	• ReAct agent from <code>create_react_agent()</code> • OpenAI Functions / Tools agent • AgentExecutor: verbose mode, max iterations • Human-in-the-loop interrupts • Agent observability with LangSmith
---	-------------------------	---

7B — LangGraph: Stateful AI Agents & Multi-Agent Systems

#	Topic	Key Concepts & Sub-topics
1	LangGraph Fundamentals	• What is LangGraph? Graph-based agent architecture • StateGraph vs. MessageGraph • Nodes, Edges, and Conditional Edges • TypedDict for state schema definition • Compiling and invoking a graph
2	Graph State & Flow	• State as a TypedDict — reducers and annotations • <code>add_messages</code> annotation for conversation history • Conditional routing with <code>conditional_edges()</code> • START and END node constants • Streaming graph execution (<code>stream()</code>, <code>astream()</code>)
3	Tool Integration	• ToolNode for automatic tool calling in graphs • <code>tools_condition</code> for routing to tool node • Binding tools to LLM nodes • Parallel tool execution in graphs • Tool results handling and state updates
4	Human-in-the-Loop	• Interrupt before/after node execution • Resuming graph from interrupt with <code>Command(resume=...)</code> • Approval workflows for high-stakes tool calls • Human feedback injection into state • Breakpoints for debugging graphs
5	Persistence & Memory	• MemorySaver for in-memory checkpointing • SQLiteSaver and PostgresSaver for persistent state • Thread-based conversation management (<code>thread_id</code>) • Cross-conversation long-term memory • State versioning and rollback
6	Multi-Agent Systems	• Supervisor agent architecture • Handoff between specialized agents • Parallel agent execution with <code>Send()</code> API • Shared memory across agents • Subgraph composition for modular multi-agent systems
7	Advanced Patterns	• Plan-and-Execute agent pattern • Reflection and Self-Critique agent loops • Map-Reduce over documents with LangGraph • Dynamic graph construction • Error recovery and retry nodes in graphs
8	LangGraph Platform	• LangGraph Studio for visual graph debugging

	• LangGraph Server for production deployment (API) • LangGraph Cloud (managed hosting) • Streaming with Server-Sent Events (SSE) • Deployment on AWS / GCP with LangGraph Server
--	--

 Mini Project

Build a Research Agent with LangGraph — takes a user query, plans sub-tasks, searches the web (Tavily), reads articles, composes a detailed report, and stores research in memory across sessions.

PHASE 8 Model Context Protocol (MCP) & Production Deployment

3–4 Weeks

MCP is the emerging open standard for connecting LLMs to external tools and data sources. This phase also covers production readiness — packaging, deploying, and monitoring AI applications.

8A — Model Context Protocol (MCP)

#	Topic	Key Concepts & Sub-topics
1	MCP Fundamentals	- What is MCP? The 'USB-C for AI' analogy - MCP vs. Function Calling vs. REST APIs - MCP Architecture: Host, Client, Server - Transport protocols: stdio, SSE (HTTP+SSE) - MCP specification overview (tools, resources, prompts, sampling)
2	MCP Primitives	- Tools: defining callable functions for LLMs - Resources: exposing data (files, DB rows, API data) - Prompts: reusable prompt templates via MCP - Sampling: server-initiated LLM calls - Roots: filesystem boundary declaration
3	Building MCP Servers	- MCP Python SDK installation and setup - Creating a basic MCP server with <code>@mcp.tool()</code> - Defining tool schemas with Pydantic - Implementing resource endpoints with <code>@mcp.resource()</code> - Adding MCP prompts with <code>@mcp.prompt()</code>
4	MCP Client Integration	- Connecting MCP server to Claude Desktop - MCP client in Python (<code>mcp.ClientSession</code>) - LangChain MCP integration (<code>langchain-mcp-adapters</code>) - Multi-server MCP client for parallel tool access - Testing MCP servers with MCP Inspector
5	Real-world MCP Servers	- File system MCP server — read/write local files - Database MCP server — query SQLite/PostgreSQL - Web search MCP server — Brave/Tavily integration - GitHub MCP server — repos, issues, PRs - Slack / Email MCP server for agentic workflows
6	MCP + LangGraph	- Loading MCP tools into LangGraph agents - MultiServerMCPClient for connecting multiple servers - Async MCP tool calling in LangGraph nodes - Streaming MCP tool results - Production patterns: MCP server pool management

8B — Production Deployment & MLOps

#	Topic	Key Concepts & Sub-topics
1	API Development	- FastAPI for serving GenAI applications - Async endpoints for LLM streaming - Pydantic request/response models - CORS, authentication middleware - API versioning and documentation with Swagger
2	Containerization	- Docker fundamentals: images, containers, Dockerfile - Writing efficient Dockerfiles for Python apps - docker-compose for multi-service apps (app + vector DB) - Environment variable management (.env, secrets) - Docker best practices for AI workloads
3	Monitoring & Observability	- LangSmith for LLM trace logging and debugging - Prometheus + Grafana for app metrics - Logging with structlog / loguru - Cost tracking and token usage dashboards - Alerting on LLM latency and error rates
4	Cloud Deployment	- AWS deployment: EC2, Lambda, ECS, SageMaker - GCP: Cloud Run for containerized apps, Vertex AI - Azure OpenAI Service and Azure AI Studio - Serverless LLM inference (Replicate, Modal, Together AI) - Load balancing and auto-scaling for AI APIs
5	Security & Safety	- Input validation and prompt injection prevention - PII detection and redaction (presidio library) - Rate limiting and API key management - Content moderation (OpenAI Moderation API) - Responsible AI: bias testing, fairness evaluation
6	GenAI CI/CD	- GitHub Actions for automated testing and deployment - Evals in CI: run prompt tests on every commit - Model versioning with DVC and MLflow - Blue-green deployment for model updates - Rollback strategies for LLM-based services



Capstone Project

Build a full-stack AI assistant with: LangGraph agent backend + RAG knowledge base + MCP server for tool access + FastAPI REST API + Docker deployment + LangSmith monitoring.

Industry Tools & Ecosystem Reference

Category	Primary Tools	Secondary / Alternatives
LLM Providers	OpenAI GPT-4o, Anthropic Claude 3.5	Google Gemini, Mistral, Cohere, Llama 3
Local LLMs	Ollama, LM Studio	llama.cpp, vLLM, Hugging Face TGI
Vector DBs	ChromaDB, FAISS	Pinecone, Qdrant, Weaviate, pgvector
AI Frameworks	LangChain v0.3, LangGraph	LlamaIndex, Haystack, CrewAI
Experiment Tracking	MLflow, W&B (Weights & Biases)	Neptune.ai, Comet ML
Prompt Evaluation	RAGAs, LangSmith, Promptfoo	TruLens, DeepEval, MT-Bench
MCP	MCP Python SDK, Claude Desktop	LangChain MCP Adapters, MCP Inspector
API Framework	FastAPI, Uvicorn	Flask, Django REST, LitServe
Containerization	Docker, docker-compose	Podman, Kubernetes (advanced)
Cloud AI	AWS SageMaker, Azure AI Studio	GCP Vertex AI, Replicate, Modal
Monitoring	LangSmith, Prometheus + Grafana	Datadog, New Relic, Helicone
IDE & Tools	VS Code + Copilot, Jupyter Lab	PyCharm, Google Colab, Cursor

Recommended Projects Portfolio

Each project below should be implemented end-to-end and pushed to GitHub with proper documentation.

#	Project Name	Key Skills Demonstrated	Difficulty
1	CLI Data Analysis Tool	Python, Pandas, Matplotlib, argparse	★ Beginner
2	Customer Churn Predictor	ML Pipeline, XGBoost, SHAP, MLflow	★ ★ Beginner+
3	Sentiment Analysis API	BERT, HuggingFace, FastAPI	★ ★ Intermediate
4	PDF RAG Chatbot	RAG, ChromaDB, LangChain, OpenAI	★ ★ ★ Intermediate
5	Prompt Optimization Suite	Prompt Engineering, LLM-as-Judge, RAGAs	★ ★ ★ Intermediate
6	AI Research Agent	LangGraph, ReAct, Tavily, Memory	★ ★ ★ Intermediate
7	Custom MCP Server	MCP SDK, Tools, Resources, Claude Desktop	★ ★ ★ Intermediate
8	Multi-Agent Code Reviewer	LangGraph Multi-Agent, Subgraphs, GitHub API	★ ★ ★ ★ Advanced
9	Conversational BI Dashboard	Text-to-SQL, LangChain, Plotly, FastAPI	★ ★ ★ ★ Advanced
10	Fine-tuned Domain LLM	LoRA, QLoRA, trl, HuggingFace Trainer	★ ★ ★ ★ Advanced
11	Enterprise AI Assistant	LangGraph + MCP + RAG + FastAPI + Docker	★ ★ ★ ★ ★ Capstone
12	AI Safety Evaluator	Prompt injection tests, bias eval, RAGAs CI	★ ★ ★ ★ ★ Capstone

Study Tips, Resources & Weekly Schedule

Recommended Learning Resources

Books

- Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow — Aurélien Géron
- Deep Learning — Goodfellow, Bengio, Courville (free online)
- Natural Language Processing with Transformers — Lewis Tunstall (HuggingFace)
- Building LLM Applications — Valentina Alto

Online Courses

- fast.ai Practical Deep Learning — free, hands-on, Python-first
- DeepLearning.AI Short Courses — LangChain, RAG, LangGraph, Prompt Engineering
- Andrej Karpathy: Neural Networks: Zero to Hero (YouTube) — transformers from scratch
- LangChain Academy — LangGraph official course (free)
- Coursera: Machine Learning Specialization — Andrew Ng

Online Platforms & Documentation

- LangChain Docs: python.langchain.com — primary reference for LangChain/LangGraph
- HuggingFace Learn: huggingface.co/learn — NLP course, audio, vision
- MCP Docs: modelcontextprotocol.io — official MCP specification
- OpenAI Cookbook: github.com/openai/openai-cookbook
- Towards Data Science & Papers With Code — staying current

Suggested Weekly Schedule

Day / Time	Activity
Mon–Fri Morning (2 hrs)	Theory study: read docs, watch tutorials, understand concepts
Mon–Fri Evening (2–3 hrs)	Hands-on coding: implement the day's concept in a Jupyter notebook
Saturday (4 hrs)	Mini-project work or solving LeetCode / HackerRank Python challenges
Sunday (2 hrs)	Review the week: write a summary note, push code to GitHub, revise flashcards

Assessment & Progression Criteria

The trainee should demonstrate the following before moving to the next phase:

- Phase 1 → 2: Can write clean Python OOP code independently. Mini-project submitted to GitHub.
- Phase 2 → 3: Understands gradient descent and matrix ops. Can derive loss functions.
- Phase 3 → 4: Built and evaluated an end-to-end ML model. MLflow experiment tracked.
- Phase 4 → 5: Can fine-tune a HuggingFace model. Understands attention mechanism.
- Phase 5 → 6: Built a working RAG pipeline. Can explain chunking and embedding strategies.
- Phase 6 → 7: Designed and evaluated 3+ prompt variants using systematic evaluation.
- Phase 7 → 8: Built a LangGraph agent with tool calling, memory, and human-in-the-loop.
- Completion: Full capstone project deployed, documented, and presented.



Ready to Build the Future with AI

This curriculum is a living document. Update it every 6 months — the GenAI space moves fast!